

Assignment 5

การออกแบบระบบและต้นแบบผู้จัดทำ

นางสาว สุธิดา ป้อมสุวรรณ 67122250057

นางสาว นริศรา อินจันทร์ 67122250047

เสนอ

ผู้ช่วยศาสตราจารย์ ดร. เสถียร จันทร์ปลา

รายงานนี้เป็นส่วนหนึ่งของวิชา CSD3202-002 การพัฒนาเว็บแอปพลิเคชัน

สาขาวิชาวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

คณะวิทยาศาสตร์และเทคโนโลยี

มหาวิทยาลัยราชภัฏสวนสุนันทา

ภาคเรียนที่ 1 ปีการศึกษา 2569

ระบบจองบัตรคอนเสิร์ต

TicketHub – Concert Ticket Booking System

ปัญหาหรือความต้องการของระบบ

การจองบัตรคอนเสิร์ตจำเป็นต้องใช้ข้อมูลหลายส่วน เช่น ข้อมูลคอนเสิร์ต รอบการแสดง ประเภทบัตร ราคา และที่นั่ง หากผู้ใช้งานต้องค้นหาข้อมูลจากหลายช่องทาง อาจทำให้เกิดความไม่สะดวกและใช้เวลาในการตัดสินใจจอง

ระบบ TicketHub จึงถูกออกแบบเป็นเว็บแอปพลิเคชันสำหรับรวบรวมข้อมูลคอนเสิร์ตและสนับสนุนกระบวนการจองบัตรไว้ในระบบเดียว ผู้ใช้งานสามารถค้นหาคอนเสิร์ต ดูรายละเอียด เลือกรอบการแสดง เลือกประเภทบัตร เลือกที่นั่ง และยืนยันการจองได้ตามลำดับ

นอกจากนี้ ระบบยังรองรับการจัดการข้อมูลสำหรับผู้จัดงานและผู้ดูแลระบบ เพื่อให้สามารถจัดการข้อมูลคอนเสิร์ต รอบการแสดง ประเภทบัตร รายการจอง และข้อมูลผู้ใช้งานได้อย่างเป็นระบบ

กลุ่มผู้ใช้งานเป้าหมาย

ระบบ TicketHub แบ่งผู้ใช้งานออกเป็น 3 กลุ่ม ดังนี้

1. Customer (ลูกค้า)

เป็นผู้ใช้งานที่ต้องการค้นหาและจองบัตรคอนเสิร์ต สามารถสมัครสมาชิก เข้าสู่ระบบ ค้นหาคอนเสิร์ต ดูรายละเอียด เลือกรอบการแสดง เลือกประเภทบัตร เลือกที่นั่ง ยืนยันการจอง และตรวจสอบประวัติการจองได้

2. Organizer (ผู้จัดงาน)

เป็นผู้ที่รับผิดชอบการจัดการข้อมูลคอนเสิร์ต สามารถเพิ่ม แก้ไข และจัดการข้อมูลคอนเสิร์ต รอบการแสดง ประเภทบัตร รวมถึงตรวจสอบรายการจอง

3. Admin (ผู้ดูแลระบบ)

เป็นผู้ที่ดูแลข้อมูลผู้ใช้งานและจัดการสถานะบัญชีภายในระบบ เพื่อให้ระบบสามารถควบคุมสิทธิ์การเข้าถึงของผู้ใช้งานแต่ละประเภทได้

วัตถุประสงค์ของระบบ

- เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับค้นหาและจองบัตรคอนเสิร์ต
- เพื่ออำนวยความสะดวกให้ผู้ใช้งานสามารถค้นหาและดูรายละเอียดคอนเสิร์ตได้จากระบบเดียว
- เพื่อให้ผู้ใช้งานสามารถเลือกรอบการแสดง ประเภทบัตร และที่นั่งที่ต้องการได้
- เพื่อจัดเก็บข้อมูลการจองของผู้ใช้งานอย่างเป็นระบบ
- เพื่อสนับสนุนผู้จัดงานในการจัดการข้อมูลคอนเสิร์ต รอบการแสดง ประเภทบัตร และรายการจอง
- เพื่อสนับสนุนผู้ดูแลระบบในการจัดการข้อมูลผู้ใช้งานและสถานะบัญชี

ขอบเขตการทำงานของระบบ

ขอบเขตของระบบ TicketHub ครอบคลุมกระบวนการตั้งแต่การเข้าสู่ระบบ การค้นหาคอนเสิร์ต ไปจนถึงการจองบัตรและตรวจสอบประวัติการจอง โดยสามารถแบ่งตามผู้ใช้งานได้ดังนี้

Customer

- สมัครสมาชิก
- เข้าสู่ระบบ
- ค้นหาคอนเสิร์ต
- ดูรายการคอนเสิร์ต
- ดูรายละเอียดคอนเสิร์ต
- เลือกรอบการแสดง
- เลือกประเภทบัตร
- เลือกที่นั่ง
- ยืนยันการจอง
- ตรวจสอบประวัติการจอง

Organizer

- เข้าสู่ระบบ
- ดู Dashboard
- จัดการข้อมูลคอนเสิร์ต
- จัดการรอบการแสดง
- จัดการประเภทบัตร
- ตรวจสอบรายการจอง

Admin

- เข้าสู่ระบบ
- ดูข้อมูลผู้ใช้งาน
- แก้ไขข้อมูลผู้ใช้งาน
- จัดการสถานะบัญชีผู้ใช้งาน

การจำแนกฟังก์ชันของระบบ

เพื่อกำหนดขอบเขตการพัฒนา ฟังก์ชันของ TicketHub แบ่งออกเป็น 3 ระดับ ได้แก่ Core Features, Supporting Features และ Future Features

Core Features

ฟังก์ชันที่จำเป็นต่อกระบวนการหลักของระบบจองบัตรคอนเสิร์ต ได้แก่

รหัส	ฟังก์ชัน	ผู้ใช้งาน
F01	สมัครสมาชิก	Customer
F02	เข้าสู่ระบบ	Customer
F03	ค้นหาคอนเสิร์ต	Customer
F04	แสดงรายการคอนเสิร์ต	Customer
F05	แสดงรายละเอียดคอนเสิร์ต	Customer
F06	เลือกรอบการแสดง	Customer
F07	เลือกประเภทบัตร	Customer
F08	เลือกที่นั่ง	Customer
F09	ยืนยันการจอง	Customer
F10	ตรวจสอบประวัติการจอง	Customer

Supporting Features

ฟังก์ชันที่ช่วยสนับสนุนการบริหารจัดการระบบ ได้แก่

รหัส	ฟังก์ชัน	ผู้ใช้งาน
F11	จัดการข้อมูลคอนเสิร์ต	Organizer
F12	จัดการรอบการแสดง	Organizer
F13	จัดการประเภทบัตร	Organizer
F14	ตรวจสอบรายการจอง	Organizer
F15	จัดการข้อมูลผู้ใช้งาน	Admin

Future Features

ฟังก์ชันที่สามารถพัฒนาเพิ่มเติมในอนาคต ได้แก่

1. ระบบชำระเงินออนไลน์
2. ระบบสร้างและตรวจสอบ QR Code สำหรับบัตรเข้าชม
3. ระบบแจ้งเตือนสถานะการจอง
4. ระบบแนะนำคอนเสิร์ตตามความสนใจของผู้ใช้งาน

ฟังก์ชันเหล่านี้ยังไม่รวมอยู่ในขอบเขตหลักของการพัฒนาครั้งนี้ เนื่องจากสามารถพิจารณาเพิ่มเติมหลังจากระบบพื้นฐานสามารถทำงานได้ครบถ้วน

ตารางรายละเอียดฟังก์ชัน

รหัส	ฟังก์ชัน	ผู้ใช้งาน	ข้อมูลนำเข้า	ผลลัพธ์	ระดับ
F01	สมัครสมาชิก	Customer	ชื่อ, Email, Password, เบอร์โทรศัพท์	บัญชีผู้ใช้งาน	Core
F02	เข้าสู่ระบบ	Customer	Email, Password	ผลการเข้าสู่ระบบ	Core
F03	ค้นหาคอนเสิร์ต	Customer	คำค้นหา	รายการคอนเสิร์ต	Core
F04	แสดงรายการคอนเสิร์ต	Customer	ข้อมูลคอนเสิร์ต	รายการคอนเสิร์ต	Core
F05	ดูรายละเอียดคอนเสิร์ต	Customer	Concert ID	รายละเอียดคอนเสิร์ต	Core
F06	เลือกรอบการแสดง	Customer	Concert ID	รายการรอบการแสดง	Core
F07	เลือกประเภทบัตร	Customer	Concert ID	ประเภทบัตรและราคา	Core
F08	เลือกที่นั่ง	Customer	Show ID, Seat ID	ที่นั่งที่เลือก	Core
F09	ยืนยันการจอง	Customer	User ID, Concert ID, Show ID, Ticket Type ID, Seat ID	ข้อมูลการจอง	Core
F10	ตรวจสอบประวัติการจอง	Customer	User ID	รายการจอง	Core
F11	จัดการคอนเสิร์ต	Organizer	ข้อมูลคอนเสิร์ต	ข้อมูลคอนเสิร์ตที่เพิ่ม/แก้ไข	Supporting
F12	จัดการรอบการแสดง	Organizer	วันที่, เวลา, สถานที่	ข้อมูลรอบการแสดง	Supporting
F13	จัดการประเภทบัตร	Organizer	ประเภทบัตร, ราคา, จำนวน	ข้อมูลประเภทบัตร	Supporting
F14	ตรวจสอบรายการจอง	Organizer	Booking ID	รายการจอง	Supporting
F15	จัดการผู้ใช้งาน	Admin	User ID, Role, Status	ข้อมูผู้ใช้งาน	Supporting

ความสัมพันธ์ระหว่างฟังก์ชันกับข้อมูล Firebase

ฟังก์ชันที่กำหนดในส่วนนี้ถูกนำไปใช้ในการออกแบบโครงสร้างข้อมูล Firebase ในส่วนที่ 2 ดังนี้

ฟังก์ชัน	Firebase Collection
สมัครสมาชิก	users
เข้าสู่ระบบ	users
ค้นหาคอนเสิร์ต	concerts
แสดงรายการคอนเสิร์ต	concerts
ดูรายละเอียดคอนเสิร์ต	concerts
เลือกรอบการแสดง	shows
เลือกประเภทบัตร	ticketTypes
เลือกที่นั่ง	seats
ยืนยันการจอง	bookings
ตรวจสอบประวัติการจอง	bookings
จัดการคอนเสิร์ต	concerts
จัดการรอบการแสดง	shows
จัดการประเภทบัตร	ticketTypes
ตรวจสอบรายการจอง	bookings
จัดการผู้ใช้งาน	users

สรุปขอบเขตของระบบ

ระบบ TicketHub เป็นเว็บแอปพลิเคชันสำหรับค้นหาและจองบัตรคอนเสิร์ต โดยมี Customer เป็นผู้ใช้งานหลักในกระบวนการค้นหา เลือกคอนเสิร์ต เลือกรอบการแสดง ประเภทบัตร และที่นั่ง และยืนยันการจอง

ระบบยังรองรับการทำงานของ Organizer ในการจัดการข้อมูลคอนเสิร์ต รอบการแสดง ประเภทบัตร และรายการจอง รวมถึงรองรับ Admin ในการจัดการข้อมูลผู้ใช้งาน

การกำหนดฟังก์ชันในส่วนนี้ถูกนำไปใช้เป็นพื้นฐานในการออกแบบโครงสร้างข้อมูล Firebase ในส่วนที่ 2 และนำไปกำหนดหน้าจอและ Wireframe ในส่วนที่ 3 จากนั้นจึงนำ Wireframe ไปวิเคราะห์และออกแบบ React Component, Routing และ Folder Structure ในส่วนที่ 4 เพื่อให้ผลลัพธ์ทั้ง 4 ส่วนมีความสัมพันธ์และสามารถนำไปใช้พัฒนาระบบ TicketHub ได้จริง

การออกแบบโครงสร้างข้อมูล Firebase

แนวทางการออกแบบโครงสร้างข้อมูล

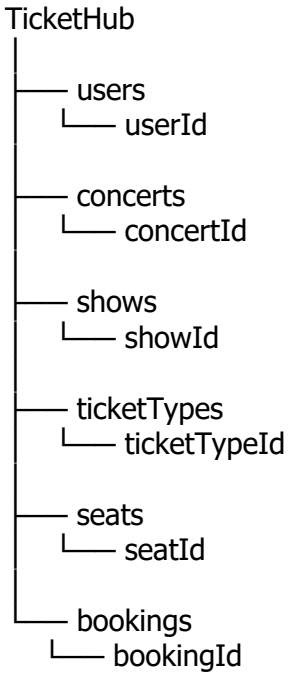
ระบบ TicketHub ออกแบบโครงสร้างข้อมูลโดยใช้ Firebase เป็นแหล่งจัดเก็บข้อมูลหลักของระบบ เพื่อรองรับการทำงานของผู้ใช้งานแต่ละประเภท ได้แก่ Customer, Organizer และ Admin

โครงสร้างข้อมูลถูกแบ่งออกเป็น 6 Collection หลัก ได้แก่ users, concerts, shows, ticketTypes, seats และ bookings โดยแต่ละ Collection มีหน้าที่จัดเก็บข้อมูลเฉพาะด้าน และใช้รหัสประจำข้อมูล เช่น userId, concertId, showId, ticketTypeId, seatId และ bookingId สำหรับเชื่อมโยงข้อมูลระหว่างกัน

การออกแบบดังกล่าวช่วยให้ข้อมูลมีโครงสร้างชัดเจน สามารถเรียกใช้ข้อมูลจาก React Component ได้ตรงตามหน้าที่ และสามารถนำโครงสร้างข้อมูลไปพัฒนาระบบจริงได้

โครงสร้าง Collection ของ Firebase

โครงสร้างข้อมูลของระบบ TicketHub มีดังนี้



โดยแต่ละ Collection มีรายละเอียดดังนี้

Collection	หน้าที่
users	จัดเก็บข้อมูล Customer, Organizer และ Admin
concerts	จัดเก็บข้อมูลคอนเสิร์ต
shows	จัดเก็บข้อมูลรอบการแสดง
ticketTypes	จัดเก็บประเภทบัตร ราคา และจำนวนบัตร
seats	จัดเก็บข้อมูลที่นั่งและสถานะที่นั่ง
bookings	จัดเก็บข้อมูลการจองของ Customer

Collection users

ใช้จัดเก็บข้อมูลผู้ใช้งานระบบ โดยกำหนด role เพื่อระบุสิทธิ์ของผู้ใช้งาน

ตัวอย่างข้อมูล:


```
{
  "userId": "U001",
  "name": "Somchai Jaidee",
  "email": "somchai@example.com",
  "phone": "0812345678",
  "role": "customer",
  "status": "active"
}
```


ข้อมูลสำคัญของ Collection นี้ ได้แก่

- userId รหัสผู้ใช้งาน
- name ชื่อผู้ใช้งาน
- email อีเมล
- phone เบอร์โทรศัพท์
- role บทบาทของผู้ใช้งาน
- status สถานะบัญชี

ค่า role สามารถกำหนดเป็น customer, organizer หรือ admin

Collection concerts

ใช้จัดเก็บข้อมูลพื้นฐานของคอนเสิร์ต


```
{
  "concertId": "C001",
  "name": "Summer Music Festival 2026",
  "artist": "Example Band",
  "description": "เทศกาลดนตรีประจำปี",
  "venue": "Bangkok Arena",
  "image": "concert01.jpg",
  "status": "available"
}
```


ข้อมูลประกอบด้วย

- concertId รหัสคอนเสิร์ต
- name ชื่อคอนเสิร์ต
- artist ศิลปิน
- description รายละเอียด
- venue สถานที่จัดงาน
- image รูปภาพ
- status สถานะคอนเสิร์ต

ข้อมูลจาก Collection นี้จะถูกนำไปใช้กับ ConcertList, ConcertCard และ ConcertInfo

Collection shows

ใช้จัดเก็บข้อมูลรอบการแสดงของคอนเสิร์ต


```
{
  "showId": "S001",
  "concertId": "C001",
  "date": "2026-12-20",
  "time": "18:00",
  "venue": "Bangkok Arena",
  "status": "available"
}
```


โดย concertId ใช้เชื่อมโยงรอบการแสดงกับคอนเสิร์ตที่เกี่ยวข้อง

ข้อมูลประกอบด้วย

- showId รหัสรอบการแสดง
- concertId รหัสคอนเสิร์ต
- date วันที่แสดง
- time เวลาแสดง
- venue สถานที่
- status สถานะรอบการแสดง

ข้อมูลนี้ถูกนำไปใช้กับ ShowList และ ShowCard

Collection ticketTypes

ใช้จัดเก็บข้อมูลประเภทบัตรของแต่ละคอนเสิร์ต


```
{
  "ticketTypeId": "T001",
  "concertId": "C001",
  "name": "VIP",
  "price": 3500,
  "quantity": 100
}
```


ข้อมูลประกอบด้วย

- ticketTypeId รหัสประเภทบัตร
- concertId รหัสคอนเสิร์ต
- name ชื่อประเภทบัตร
- price ราคาบัตร
- quantity จำนวนบัตร

ข้อมูลนี้ถูกนำไปใช้กับ TicketTypeList และ TicketTypeCard

Collection seats

ใช้จัดเก็บข้อมูลที่นั่งสำหรับแต่ละรอบการแสดง


```
{
  "seatId": "SE001",
  "showId": "S001",
  "zone": "VIP",
  "seatNumber": "A01",
  "price": 3500,
  "status": "available"
}
```


ข้อมูลประกอบด้วย

- seatId รหัสที่นั่ง
- showId รหัสรอบการแสดง
- zone โซนที่นั่ง
- seatNumber หมายเลขที่นั่ง
- price ราคาที่นั่ง
- status สถานะที่นั่ง

ค่า status สามารถใช้ available สำหรับที่นั่งว่าง และ booked สำหรับที่นั่งที่ถูกจองแล้ว

ข้อมูลนี้ถูกนำไปใช้กับ SeatMap และ Seat

Collection bookings

ใช้จัดเก็บข้อมูลการจองบัตรของ Customer


```
{
  "bookingId": "B001",
  "userId": "U001",
  "concertId": "C001",
  "showId": "S001",
  "ticketTypeId": "T001",
  "seatId": "SE001",
  "quantity": 1,
  "totalPrice": 3500,
  "bookingDate": "2026-08-30",
  "status": "confirmed"
}
```


ข้อมูลประกอบด้วย

- bookingId รหัสการจอง
- userId รหัสผู้จอง
- concertId รหัสคอนเสิร์ต
- showId รหัสรอบการแสดง
- ticketTypeId รหัสประเภทบัตร
- seatId รหัสที่นั่ง
- quantity จำนวนบัตร
- totalPrice ราคารวม
- bookingDate วันที่ทำรายการ
- status สถานะการจอง

ข้อมูลนี้ถูกนำไปใช้กับ BookingList, BookingCard และ BookingSummary

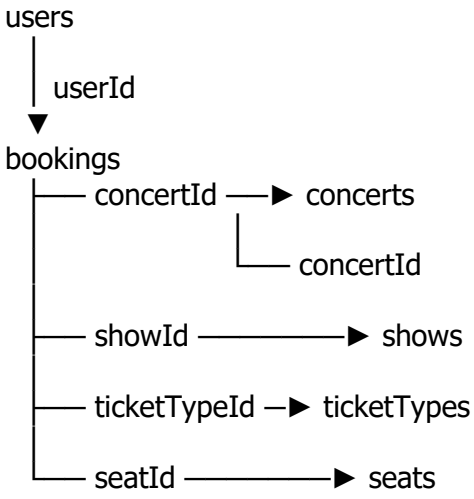
Data Dictionary

Collection	Field	Data Type	รายละเอียด
users	userId	String	รหัสผู้ใช้งาน
users	name	String	ชื่อผู้ใช้งาน
users	email	String	อีเมล
users	phone	String	เบอร์โทรศัพท์
users	role	String	บทบาทของผู้ใช้งาน
users	status	String	สถานะบัญชี
concerts	concertId	String	รหัสคอนเสิร์ต
concerts	name	String	ชื่อคอนเสิร์ต
concerts	artist	String	ชื่อศิลปิน
concerts	description	String	รายละเอียดคอนเสิร์ต
concerts	venue	String	สถานที่จัดงาน
concerts	image	String	รูปภาพคอนเสิร์ต
concerts	status	String	สถานะคอนเสิร์ต
shows	showId	String	รหัสรอบการแสดง
shows	concertId	String	รหัสคอนเสิร์ต
shows	date	String	วันที่แสดง
shows	time	String	เวลาแสดง
shows	venue	String	สถานที่จัดงาน
shows	status	String	สถานะรอบการแสดง
ticketTypes	ticketTypeId	String	รหัสประเภทบัตร
ticketTypes	concertId	String	รหัสคอนเสิร์ต
ticketTypes	name	String	ประเภทบัตร
ticketTypes	price	Number	ราคาบัตร
ticketTypes	quantity	Number	จำนวนบัตร
seats	seatId	String	รหัสที่นั่ง
seats	showId	String	รหัสรอบการแสดง
seats	zone	String	โซนที่นั่ง
seats	seatNumber	String	หมายเลขที่นั่ง
seats	price	Number	ราคาที่นั่ง
seats	status	String	สถานะที่นั่ง

bookings	bookingId	String	รหัสการจอง
bookings	userId	String	รหัสผู้จอง
bookings	concertId	String	รหัสคอนเสิร์ต
bookings	showId	String	รหัสรอบการแสดง
bookings	ticketTypeId	String	รหัสประเภทบัตร
bookings	seatId	String	รหัสที่นั่ง
bookings	quantity	Number	จำนวนบัตร
bookings	totalPrice	Number	ราคารวม
bookings	bookingDate	String	วันที่จอง
bookings	status	String	สถานะการจอง

ความสัมพันธ์ระหว่างข้อมูล

ข้อมูลในระบบมีความสัมพันธ์กันผ่านรหัสอ้างอิง ดังนี้



ตัวอย่างเช่น เมื่อ Customer ที่มี userId = U001 ทำการจองคอนเสิร์ต C001 ระบบจะบันทึก concertId = C001 ลงใน bookings และเชื่อมโยงกับรอบการแสดง S001 ประเภทบัตร T001 และที่นั่ง SE001

ทำให้ระบบสามารถตรวจสอบได้ว่า **ผู้ใช้งานคนใดจองคอนเสิร์ตใด รอบใด ประเภทบัตรใด และเลือกที่นั่งใด**

ความสัมพันธ์ระหว่าง Firebase กับ React Component

Firebase Collection	React Component	หน้าที่
users	Login, Register	จัดการข้อมูลผู้ใช้งาน
concerts	ConcertList	แสดงรายการคอนเสิร์ต
concerts	ConcertCard	แสดงข้อมูลคอนเสิร์ตแต่ละรายการ
concerts	ConcertInfo	แสดงรายละเอียดคอนเสิร์ต
shows	ShowList	แสดงรอบการแสดง
shows	ShowCard	แสดงข้อมูลรอบการแสดง
ticketTypes	TicketTypeList	แสดงประเภทบัตร
ticketTypes	TicketTypeCard	แสดงรายละเอียดประเภทบัตร
seats	SeatMap	แสดงผังที่นั่ง
seats	Seat	แสดงสถานะของที่นั่ง
bookings	BookingSummary	แสดงสรุปการจอง
bookings	BookingList	แสดงประวัติการจอง
bookings	BookingCard	แสดงรายละเอียดการจอง

สรุปการออกแบบโครงสร้างข้อมูล

โครงสร้างข้อมูล Firebase ของระบบ TicketHub แบ่งข้อมูลออกเป็น 6 Collection ได้แก่ users, concerts, shows, ticketTypes, seats และ bookings โดยแต่ละ Collection มีหน้าที่เฉพาะและสามารถเชื่อมโยงกันผ่านรหัสประจำข้อมูล

โครงสร้างดังกล่าวรองรับการทำงานตั้งแต่การสมัครสมาชิก การค้นหาและแสดงข้อมูลคอนเสิร์ต การเลือกรอบการแสดง ประเภทบัตร และที่นั่ง ไปจนถึงการบันทึกและตรวจสอบข้อมูลการจอง นอกจากนี้ยังสามารถเชื่อมโยงกับ React Component ที่ออกแบบไว้ในส่วนที่ 4 ได้โดยตรง ทำให้โครงสร้างข้อมูลและส่วนติดต่อผู้ใช้สามารถนำไปพัฒนาระบบจริงได้อย่างสอดคล้องกัน

การออกแบบส่วนติดต่อผู้ใช้และ Wireframe

แนวทางการออกแบบส่วนติดต่อผู้ใช้

การออกแบบส่วนติดต่อผู้ใช้ของระบบ TicketHub มีวัตถุประสงค์เพื่อให้ผู้ใช้งานสามารถค้นหาและจองบัตรคอนเสิร์ตได้อย่างสะดวก โดยออกแบบลำดับการใช้งานตั้งแต่การเข้าสู่ระบบ การค้นหาคอนเสิร์ต การดูรายละเอียด การเลือกรอบการแสดง การเลือกประเภทบัตร การเลือกที่นั่ง และการยืนยันการจอง

การออกแบบหน้าจอพิจารณาจากฟังก์ชันของระบบและโครงสร้างข้อมูล Firebase ในส่วนที่ 2 เพื่อให้แต่ละหน้าจอสามารถนำไปพัฒนาเป็น React Page และ Component ได้จริง

ระบบแบ่งหน้าจอตามบทบาทของผู้ใช้งาน ได้แก่ Customer, Organizer และ Admin เพื่อให้ผู้ใช้งานแต่ละประเภทเข้าถึงฟังก์ชันที่เกี่ยวข้องกับตนเอง

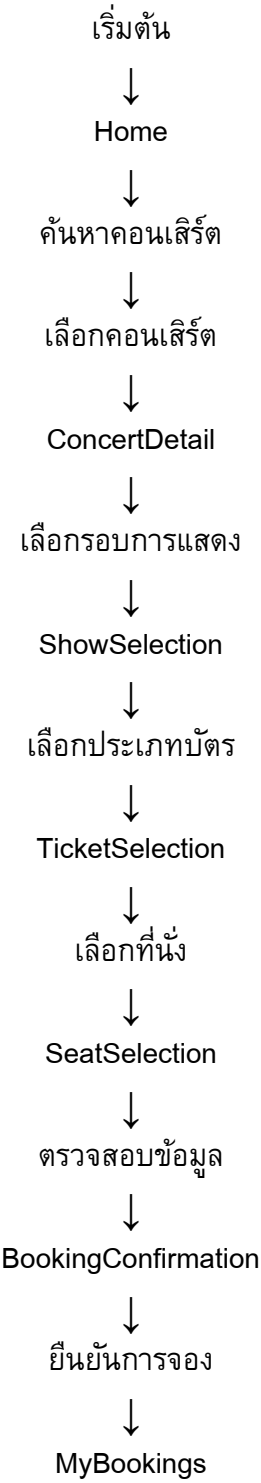
Sitemap ของระบบ

โครงสร้างหน้าเว็บของ TicketHub สามารถแสดงได้ดังนี้

```
TicketHub
├── Home
├── Authentication
│   ├── Login
│   └── Register
├── Concert
│   ├── ConcertList
│   └── ConcertDetail
│       ├── ShowSelection
│       └── TicketSelection
│           └── SeatSelection
│               └── BookingConfirmation
├── Customer
│   └── MyBookings
├── Organizer
│   ├── OrganizerDashboard
│   ├── ConcertManagement
│   ├── ShowManagement
│   ├── TicketManagement
│   └── BookingManagement
└── Admin
    └── UserManagement
```

User Flow ของ Customer

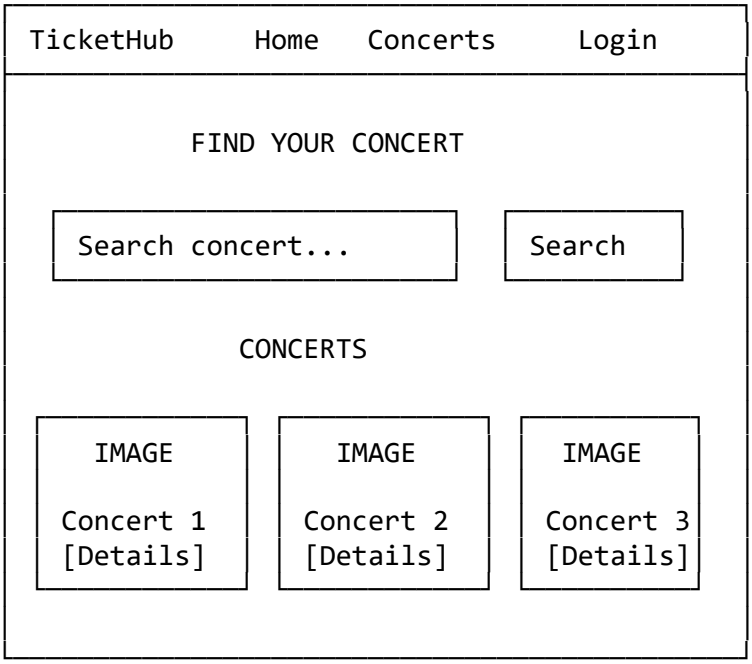
กระบวนการใช้งานหลักของ Customer มีดังนี้



User Flow ดังกล่าวสอดคล้องกับข้อมูล Firebase ที่กำหนดไว้ ได้แก่ concerts ⟶ shows ⟶ ticketTypes ⟶ seats ⟶ bookings

Wireframe หน้า Home

หน้า Home เป็นหน้าหลักของระบบ ใช้สำหรับแสดงข้อมูลเบื้องต้นและรายการคอนเสิร์ตที่เปิดให้จอง



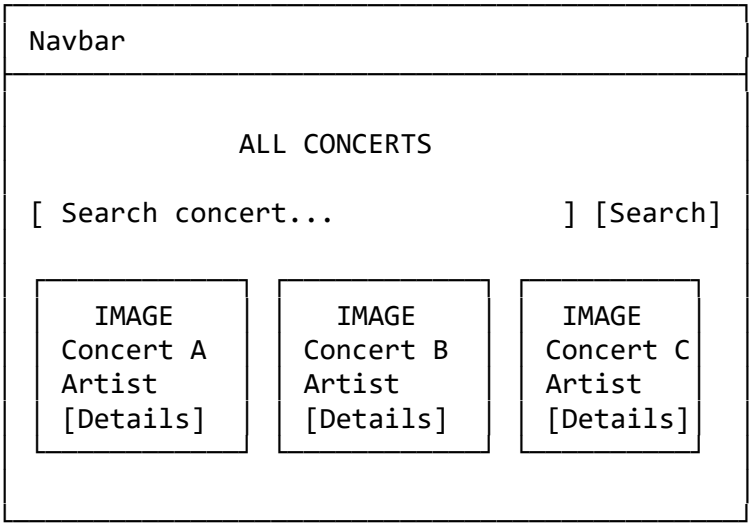
Component ที่สามารถวิเคราะห์จาก Wireframe หน้า Home ได้แก่

Home
- Navbar
- HeroSection
  - SearchBar
- ConcertList
  - ConcertCard
- Footer

ข้อมูลที่เกี่ยวข้องคือ concerts

Wireframe หน้า Concert List

หน้า Concert List ใช้สำหรับแสดงรายการคอนเสิร์ตทั้งหมด



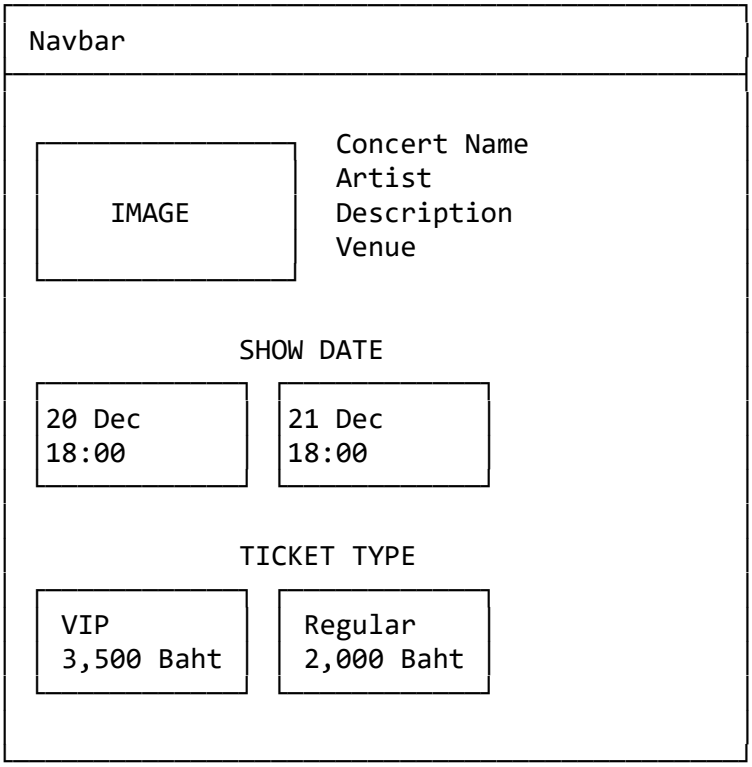
Component:

ConcertList
- Navbar
- SearchBar
- ConcertList
  - ConcertCard
- Footer

ข้อมูลจาก Firebase:

concerts ⟶ name, artist, image, status

Wireframe หน้า Concert Detail

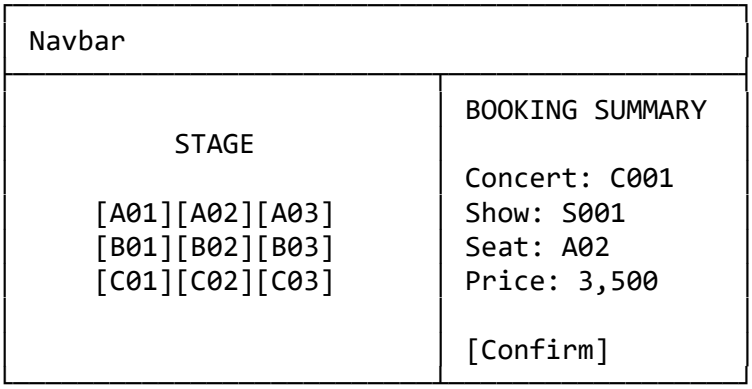


Component:
ConcertDetail
- Navbar
- ConcertInfo
- ShowList
  - ShowCard
- TicketTypeList
  - TicketTypeCard
- Footer

ข้อมูล Firebase:

- concerts
- shows
- ticketTypes

Wireframe หน้า Seat Selection



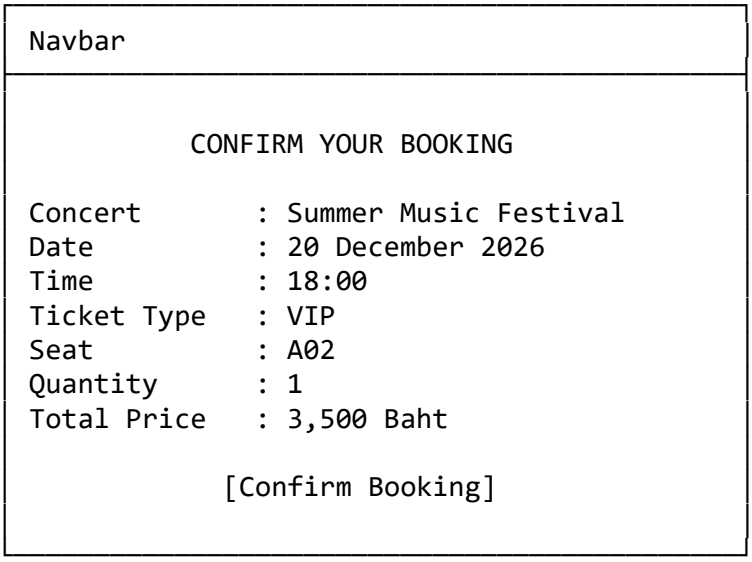
Component:
SeatSelection
- Navbar
- ConcertSummary
- SeatMap
  - Seat
- BookingSummary
- Footer

ข้อมูล Firebase:

seats ⟶ seatId, showId, zone, seatNumber, price, status

เมื่อผู้ใช้งานเลือกที่นั่ง State selectedSeat จะเก็บที่นั่งที่เลือกไว้ก่อนส่งไปยังขั้นตอนยืนยันการจอง

Wireframe หน้า Booking Confirmation



Component:
BookingConfirmation
├── Navbar
├── BookingSummary
└── Footer

เมื่อผู้ใช้งานกดยืนยัน ระบบจะสร้างข้อมูลใน Collection bookings

Wireframe หน้า My Bookings



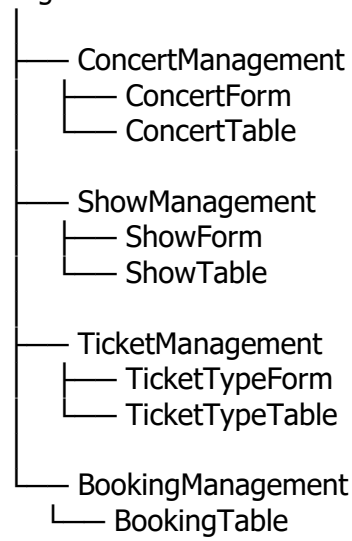
Component:
MyBookings
- Navbar
- BookingList
  - BookingCard
- Footer

ข้อมูล Firebase:

bookings ⟶ bookingId, userId, concertId, showId, seatId, totalPrice, status

Wireframe หน้าสำหรับ Organizer

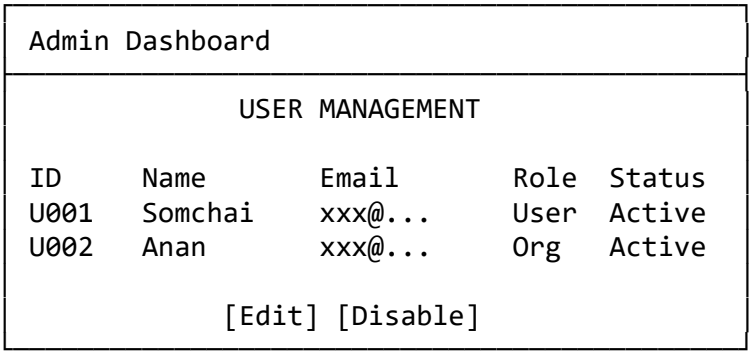
Organizer มีหน้าหลักสำหรับจัดการข้อมูลระบบ



ข้อมูลที่เกี่ยวข้องคือ concerts, shows, ticketTypes และ bookings

Wireframe หน้าสำหรับ Admin

Admin สามารถจัดการข้อมูลผู้ใช้งานผ่านหน้า User Management



ข้อมูล Firebase:

users ⟶ userId, name, email, role, status

ความสัมพันธ์ระหว่าง Wireframe กับ Firebase

หน้าจอ	Firebase Collection	Field ที่ใช้
Home	concerts	name, artist, image, status
ConcertList	concerts	name, artist, image
ConcertDetail	concerts	name, artist, description, venue, image
ShowSelection	shows	concertId, date, time, venue, status
TicketSelection	ticketTypes	concertId, name, price, quantity
SeatSelection	seats	showId, zone, seatNumber, price, status
BookingConfirmation	bookings	userId, concertId, showId, ticketTypeId, seatId, quantity, totalPrice
MyBookings	bookings	bookingId, userId, concertId, showId, seatId, totalPrice, status
ConcertManagement	concerts	ข้อมูลคอนเสิร์ตทั้งหมด
ShowManagement	shows	ข้อมูลรอบการแสดงทั้งหมด
TicketManagement	ticketTypes	ข้อมูลประเภทบัตรทั้งหมด
BookingManagement	bookings	ข้อมูลการจองทั้งหมด
UserManagement	users	userId, name, email, role, status

สรุปการออกแบบ UI และ Wireframe

การออกแบบส่วนติดต่อผู้ใช้ของระบบ TicketHub ได้กำหนดหน้าจอตามฟังก์ชันการทำงานจริงของระบบ และจัดลำดับการใช้งานของ Customer ตั้งแต่การค้นหาคอนเสิร์ตจนถึงการยืนยันการจอง

Wireframe แต่ละหน้าถูกออกแบบให้สามารถนำไปแยกเป็น React Component ได้ โดยข้อมูลที่แสดงในแต่ละหน้ามีความสัมพันธ์โดยตรงกับ Collection และ Field ที่กำหนดไว้ใน Firebase เช่น หน้า Concert Detail ใช้ข้อมูลจาก concerts, shows และ ticketTypes ขณะที่หน้า Seat Selection ใช้ข้อมูลจาก seats และหน้า My Bookings ใช้ข้อมูลจาก bookings

ดังนั้น Wireframe ที่ออกแบบในส่วนนี้สามารถนำไปใช้เป็นพื้นฐานสำหรับการกำหนด Component Hierarchy, Routing และโครงสร้าง Folder ของ React Project ในส่วนที่ 4 ได้อย่างสอดคล้องกัน

การออกแบบ Component สำหรับ React

แนวทางการออกแบบ Component

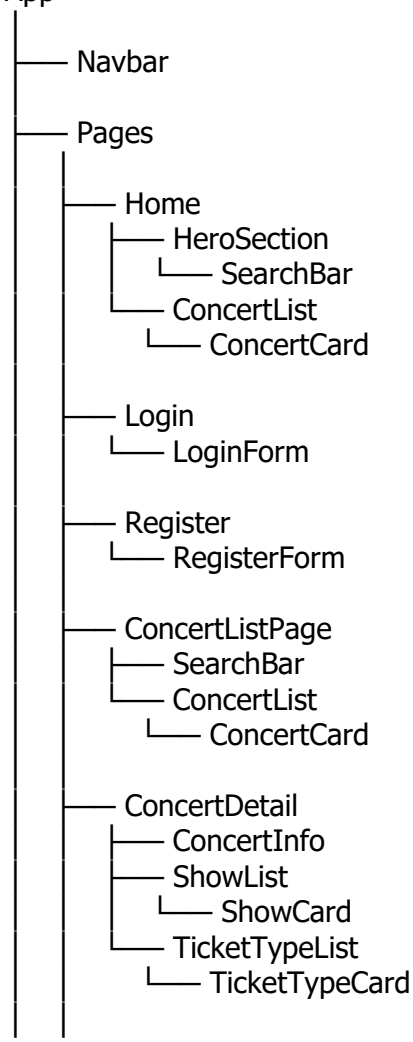
การออกแบบ Component ของระบบ TicketHub พิจารณาจาก Wireframe ที่ออกแบบไว้ในส่วนที่ 3 โดยแบ่งหน้าจอออกเป็น Page Component และ Reusable Component เพื่อให้แต่ละ Component มีหน้าที่ชัดเจนและสามารถนำกลับมาใช้ซ้ำได้

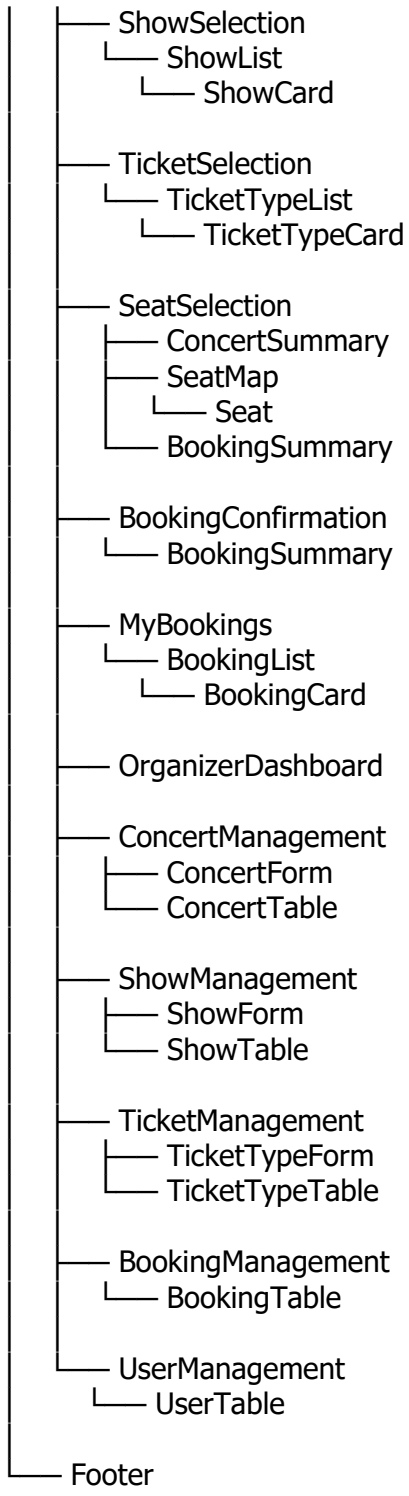
Page Component ทำหน้าที่เป็นหน้าหลักของแต่ละ Route ส่วน Reusable Component ทำหน้าที่แสดงผลหรือจัดการข้อมูลเฉพาะส่วน เช่น ConcertCard, ShowCard, TicketTypeCard, Seat และ BookingCard

การแบ่ง Component ในลักษณะนี้ช่วยลดการเขียนโค้ดซ้ำ ทำให้โครงสร้างของ React Project มีความเป็นระบบ และสามารถแก้ไขหรือพัฒนาแต่ละส่วนได้ง่าย

Component Hierarchy

โครงสร้าง Component หลักของระบบ TicketHub มีดังนี้





โครงสร้างดังกล่าวแสดงให้เห็นว่า Page Component เป็นส่วนที่ควบคุมหน้าจอหลัก ขณะที่ Component ย่อยรับผิดชอบงานเฉพาะด้านและสามารถนำไปใช้ซ้ำในหลายหน้าได้

ตารางจำแนกประเภท Component

Component	ประเภท	หน้าที่
Navbar	Shared	แสดงเมนูหลักของระบบ
Footer	Shared	แสดงส่วนท้ายของเว็บไซต์
HeroSection	Feature	แสดงส่วนแนะนำระบบในหน้า Home
SearchBar	Reusable	รับคำค้นหาคอนเสิร์ต
ConcertList	Feature	แสดงรายการคอนเสิร์ต
ConcertCard	Reusable	แสดงข้อมูลคอนเสิร์ตแต่ละรายการ
ConcertInfo	Feature	แสดงรายละเอียดคอนเสิร์ต
ShowList	Feature	แสดงรายการรอบการแสดง
ShowCard	Reusable	แสดงข้อมูลรอบการแสดง
TicketTypeList	Feature	แสดงประเภทบัตร
TicketTypeCard	Reusable	แสดงประเภทและราคาบัตร
SeatMap	Feature	แสดงผังที่นั่ง
Seat	Reusable	แสดงที่นั่งแต่ละตำแหน่ง
ConcertSummary	Feature	แสดงข้อมูลคอนเสิร์ตที่เลือก
BookingSummary	Feature	สรุปข้อมูลการจองและราคา
BookingList	Feature	แสดงรายการจอง
BookingCard	Reusable	แสดงข้อมูลการจองแต่ละรายการ
LoginForm	Feature	รับข้อมูลสำหรับเข้าสู่ระบบ
RegisterForm	Feature	รับข้อมูลสมัครสมาชิก
ConcertForm	Feature	เพิ่มหรือแก้ไขข้อมูลคอนเสิร์ต
ConcertTable	Feature	แสดงข้อมูลคอนเสิร์ตสำหรับ Organizer
ShowForm	Feature	เพิ่มหรือแก้ไขรอบการแสดง
ShowTable	Feature	แสดงข้อมูลรอบการแสดง
TicketTypeForm	Feature	เพิ่มหรือแก้ไขประเภทบัตร
TicketTypeTable	Feature	แสดงข้อมูลประเภทบัตร
BookingTable	Feature	แสดงรายการจองสำหรับ Organizer
UserTable	Feature	แสดงและจัดการข้อมูลผู้ใช้งาน

Component Specification

ตารางต่อไปนี้แสดง Props, State และข้อมูล Firebase ที่ Component แต่ละตัวเกี่ยวข้อง

Component	Props	State	Firebase
SearchBar	onSearch	keyword	concerts
ConcertList	concerts	-	concerts
ConcertCard	concert, onSelect	-	concerts
ConcertInfo	concert	-	concerts
ShowList	shows	-	shows
ShowCard	show, onSelect	-	shows
TicketTypeList	ticketTypes	-	ticketTypes
TicketTypeCard	ticketType,onSelect	-	ticketTypes
SeatMap	seats, onSelect	selectedSeat	seats
Seat	seat, onSelect	-	seats
ConcertSummary	concert, show	-	concerts, shows
BookingSummary	bookingData	totalPrice	bookings
BookingList	bookings	-	bookings
BookingCard	booking	-	bookings
LoginForm	onLogin	email, password	users
RegisterForm	onRegister	name,email,password,phone	users
ConcertForm	concert, onSave	form data	concerts
ShowForm	show, onSave	form data	shows
TicketTypeForm	ticketType,onSave	form data	ticketTypes
BookingTable	bookings	filter	bookings
UserTable	users	filter	users

การออกแบบ Props และ State

การกำหนด Props และ State พิจารณาจากหน้าที่ของแต่ละ Component โดย Props ใช้สำหรับส่งข้อมูลจาก Component แม่ไปยัง Component ลูก ส่วน State ใช้สำหรับเก็บข้อมูลที่มีการเปลี่ยนแปลงระหว่างการใช้งาน

ตัวอย่างเช่น ConcertCard รับข้อมูล concert ผ่าน Props เนื่องจากข้อมูลคอนเสิร์ตมาจาก ConcertList ส่วน SearchBar ใช้ State keyword เพื่อเก็บข้อความที่ผู้ใช้งานกำลังค้นหา สำหรับหน้า SeatSelection ใช้ State selectedSeat เพื่อเก็บที่นั่งที่ผู้ใช้งานเลือก เนื่องจากสถานะดังกล่าวเปลี่ยนแปลงตามการกระทำของผู้ใช้งาน

การกำหนด Props และ State ในลักษณะนี้ช่วยให้ข้อมูลถูกส่งระหว่าง Component อย่างเหมาะสม และลดการเก็บ State ที่ไม่จำเป็น

ตาราง Routing

กำหนด Routing ของระบบให้สอดคล้องกับ Page Component ดังนี้

Route	Page Component	ผู้ใช้งาน
/	Home	ทุกคน
/login	Login	ทุกคน
/register	Register	Customer
/concerts	ConcertListPage	ทุกคน
/concerts/:id	ConcertDetail	ทุกคน
/concerts/:id/shows	ShowSelection	Customer
/shows/:id/tickets	TicketSelection	Customer
/shows/:id/seats	SeatSelection	Customer
/booking/confirm	BookingConfirmation	Customer
/my-bookings	MyBookings	Customer
/organizer	OrganizerDashboard	Organizer
/organizer/concerts	ConcertManagement	Organizer
/organizer/shows	ShowManagement	Organizer
/organizer/tickets	TicketManagement	Organizer
/organizer/bookings	BookingManagement	Organizer
/admin/users	UserManagement	Admin

การกำหนด Route ใช้รูปแบบ PascalCase สำหรับชื่อ Component และใช้ URL ที่สื่อความหมายถึงหน้าที่ของแต่ละหน้า ทำให้สามารถเชื่อมโยง Route กับ Page Component ได้อย่างชัดเจน

โครงสร้าง Folder ของ React Project

โครงสร้าง Folder ที่เสนอสำหรับการพัฒนา TicketHub มีดังนี้

```
src/
├── components/
│   ├── common/
│   │   ├── Navbar.jsx
│   │   ├── Footer.jsx
│   │   └── SearchBar.jsx
│   ├── concert/
│   │   ├── ConcertCard.jsx
│   │   ├── ConcertList.jsx
│   │   └── ConcertInfo.jsx
│   ├── show/
│   │   ├── ShowCard.jsx
│   │   └── ShowList.jsx
│   ├── ticket/
│   │   ├── TicketTypeCard.jsx
│   │   └── TicketTypeList.jsx
│   ├── seat/
│   │   ├── Seat.jsx
│   │   └── SeatMap.jsx
│   └── booking/
│       ├── BookingCard.jsx
│       ├── BookingList.jsx
│       └── BookingSummary.jsx
├── pages/
│   ├── Home.jsx
│   ├── Login.jsx
│   ├── Register.jsx
│   ├── ConcertListPage.jsx
│   ├── ConcertDetail.jsx
│   ├── ShowSelection.jsx
│   ├── TicketSelection.jsx
```

```
│   ├── SeatSelection.jsx
│   ├── BookingConfirmation.jsx
│   ├── MyBookings.jsx
│   ├── OrganizerDashboard.jsx
│   ├── ConcertManagement.jsx
│   ├── ShowManagement.jsx
│   ├── TicketManagement.jsx
│   ├── BookingManagement.jsx
│   └── UserManagement.jsx
│
├── services/
│   └── firebase.js
│
├── App.jsx
└── main.jsx
```

การแบ่ง Folder แยกตามหน้าที่ช่วยให้สามารถค้นหาและจัดการไฟล์ได้ง่าย โดย components ใช้จัดเก็บ Component ที่นำกลับมาใช้ซ้ำ ส่วน pages ใช้จัดเก็บ Page Component ที่สัมพันธ์กับ Route และ services ใช้จัดเก็บการเชื่อมต่อกับ Firebase

เหตุผลในการแบ่ง Component

การแบ่ง Component ของ TicketHub พิจารณาจากหน้าที่ของแต่ละส่วนใน Wireframe และความสามารถในการนำกลับมาใช้ซ้ำ

Navbar, Footer และ SearchBar ถูกแยกเป็น Component เนื่องจากสามารถใช้งานร่วมกันในหลายหน้า ลดการเขียนโค้ดซ้ำ

ConcertCard, ShowCard, TicketTypeCard, Seat และ BookingCard เป็น Component ที่ใช้แสดงข้อมูลในรูปแบบรายการ จึงสามารถรับข้อมูลผ่าน Props และนำกลับมาใช้กับข้อมูลหลายรายการได้

ส่วน ConcertList, ShowList, TicketTypeList, SeatMap และ BookingList ทำหน้าที่จัดการกลุ่มข้อมูล จึงแยกออกจาก Page Component เพื่อไม่ให้ Page มีขนาดใหญ่และรับผิดชอบหลายหน้าที่เกินไป

การแบ่ง Component ดังกล่าวทำให้แต่ละส่วนมีความรับผิดชอบชัดเจน สามารถแก้ไขและทดสอบแยกส่วนได้ง่าย และสามารถเชื่อมโยงข้อมูลกับ Firebase ตามหน้าที่ของ Component ได้โดยตรง

ตารางความสัมพันธ์ระหว่างฟังก์ชัน ข้อมูล Firebase หน้าจอ และ Component

ฟังก์ชัน	Firebase	หน้าจอ React	Component
สมัครสมาชิก	users	Register	RegisterForm
เข้าสู่ระบบ	users	Login	LoginForm
ค้นหาคอนเสิร์ต	concerts	Home / ConcertList	SearchBar, ConcertList
แสดงรายการคอนเสิร์ต	concerts	ConcertList	ConcertList,ConcertCard
ดูรายละเอียดคอนเสิร์ต	concerts	ConcertDetail	ConcertInfo
เลือกรอบการแสดง	shows	ShowSelection	ShowList, ShowCard
เลือกประเภทบัตร	ticketTypes	TicketSelection	TicketTypeList,TicketTypeCard
เลือกที่นั่ง	seats	SeatSelection	SeatMap, Seat
ยืนยันการจอง	bookings	BookingConfirmation	BookingSummary
ดูประวัติการจอง	bookings	MyBookings	BookingList,BookingCard
จัดการคอนเสิร์ต	concerts	ConcertManagement	ConcertForm,ConcertTable
จัดการรอบการแสดง	shows	ShowManagement	ShowForm, ShowTable
จัดการประเภทบัตร	ticketTypes	TicketManagement	TicketTypeForm,TicketTypeTable
ตรวจสอบรายการจอง	bookings	BookingManagement	BookingTable
จัดการผู้ใช้งาน	users	UserManagement	UserTable

ตารางนี้แสดงให้เห็นว่าฟังก์ชันของระบบ ข้อมูล Firebase หน้าจอ React และ Component มีความสัมพันธ์กันตั้งแต่ต้นจนจบ ไม่ได้ออกแบบแยกออกจากกัน

ตรวจสอบการออกแบบ Component

ก่อนส่งงาน ได้ตรวจสอบการออกแบบ Component ตามข้อกำหนดดังนี้

รายการตรวจสอบ	ผลการตรวจสอบ
Wireframe ทุกหน้าสามารถแปลงเป็น Component ได้	ผ่าน
Component ที่ใช้ซ้ำได้รับการแยกออกจาก Page Component	ผ่าน
ไม่สร้าง Component ขนาดใหญ่ที่รับผิดชอบทุกอย่างในไฟล์เดียว	ผ่าน
ไม่แยก Component ย่อยมากเกินไปจนโครงสร้างซับซ้อน	ผ่าน
ชื่อ Component ใช้รูปแบบ PascalCase	ผ่าน
Page Component สอดคล้องกับ Route	ผ่าน
Props และ State มีเหตุผลรองรับ	ผ่าน
Component ที่แสดงข้อมูลสามารถเชื่อมโยงกับ Firebase ได้	ผ่าน

สรุปการออกแบบ Component

การออกแบบ Component ของระบบ TicketHub ได้วิเคราะห์จาก Wireframe ของระบบจริงและแบ่ง Component ตามหน้าที่ โดยแยก Page Component ออกจาก Reusable Component เพื่อให้โครงสร้างของระบบมีความชัดเจนและสามารถนำ Component กลับมาใช้ซ้ำได้

นอกจากนี้ Page Component แต่ละตัวถูกกำหนดให้สอดคล้องกับ Route และ Component ที่แสดงข้อมูลถูกกำหนดให้เชื่อมโยงกับ Collection และ Field ใน Firebase เช่น ConcertCard เชื่อมโยงกับข้อมูลจาก concerts, ShowCard เชื่อมโยงกับ shows, Seat เชื่อมโยงกับ seats และ BookingCard เชื่อมโยงกับ bookings

ดังนั้น การออกแบบส่วนที่ 4 มีความสัมพันธ์กับส่วนก่อนหน้าอย่างต่อเนื่อง ได้แก่ **ฟังก์ชันในส่วนที่ 1 ⟶ โครงสร้างข้อมูล Firebase ในส่วนที่ 2 ⟶ Wireframe ในส่วนที่ 3 ⟶ Component และ Routing ในส่วนที่ 4** ทำให้สามารถนำแบบจำลองทั้งหมดไปใช้เป็นแนวทางในการพัฒนาเว็บแอปพลิเคชัน TicketHub ได้จริง

งานที่ 5 การออกแบบระบบและต้นแบบ

System Design and Prototype Design

งานนี้เป็นขั้นตอนต่อเนื่องจากการจัดทำข้อเสนอโครงงาน การวิเคราะห์ Use Case และงานที่ 4 การออกแบบข้อมูลและสินทรัพย์ดิจิทัล (Data and Digital Asset Design) โดยให้นักศึกษานำผลลัพธ์จากงานที่ผ่านมาออกแบบเป็นโครงสร้างระบบที่พร้อมสำหรับการพัฒนาเว็บแอปพลิเคชันด้วย React และ Firebase

นักศึกษายังไม่ต้องพัฒนาโปรแกรมที่สมบูรณ์ในงานนี้ แต่ต้องออกแบบระบบให้เห็นความสัมพันธ์ระหว่างฟังก์ชัน ข้อมูล หน้าจอ การนำทาง และ React Component อย่างชัดเจน

วัตถุประสงค์

เมื่อนักศึกษาทำงานนี้เสร็จแล้ว นักศึกษาจะสามารถ

1. กำหนดขอบเขตและฟังก์ชันของเว็บแอปพลิเคชันให้เหมาะสมกับระยะเวลาพัฒนา
2. ออกแบบโครงสร้างข้อมูลสำหรับนำเข้า Firebase Realtime Database
3. ออกแบบโครงสร้างหน้าจอและการนำทางของเว็บแอปพลิเคชัน
4. วิเคราะห์ Wireframe และแบ่งส่วนติดต่อผู้ใช้เป็น React Component
5. จัดเตรียมแบบจำลองระบบก่อนเริ่มพัฒนาโปรแกรมจริง

สิ่งที่นักศึกษาต้องดำเนินการ

1. ทบทวนขอบเขตและฟังก์ชันของโครงงาน

ให้นักศึกษานำ Project Proposal และ Use Case ที่จัดทำไว้แล้วมาทบทวน เพื่อกำหนดขอบเขตของระบบให้สามารถพัฒนาเสร็จได้ภายในระยะเวลาที่กำหนด

1.1 ทบทวนวัตถุประสงค์ของโครงงาน

ระบุข้อมูลต่อไปนี้

- ชื่อโครงงานภาษาไทยและภาษาอังกฤษ
- ปัญหาหรือความต้องการที่โครงงานต้องการแก้ไข
- กลุ่มผู้ใช้งานเป้าหมาย
- วัตถุประสงค์ของเว็บแอปพลิเคชัน
- ประโยชน์ที่ผู้ใช้งานจะได้รับ

1.2 จำแนกฟังก์ชันของระบบ

ให้นักศึกษาแบ่งฟังก์ชันออกเป็น 3 กลุ่ม ดังนี้

กลุ่มฟังก์ชัน	ความหมาย	แนวทางดำเนินการ
Core Features	ฟังก์ชันหลักที่จำเป็นต่อวัตถุประสงค์ของระบบ	ต้องพัฒนาให้เสร็จและใช้งานได้
Supporting Features	ฟังก์ชันสนับสนุนที่ช่วยให้ระบบใช้งานได้สะดวกหรือสมบูรณ์ขึ้น	พัฒนาหลังจาก Core Features ทำงานได้แล้ว
Future Features	ฟังก์ชันที่น่าสนใจ แต่มีความซับซ้อนหรือเกินขอบเขตของรายวิชา	เสนอเป็นแนวทางพัฒนาในอนาคต

แต่ละกลุ่มควรกำหนด Core Features ประมาณ 3–5 ฟังก์ชัน ไม่ควรกำหนดฟังก์ชันจำนวนมากเกินกว่าที่จะพัฒนาและทดสอบได้จริง

1.3 จัดทำตารางรายละเอียดฟังก์ชัน

รหัส	ชื่อฟังก์ชัน	ผู้ใช้งาน	ข้อมูลนำเข้า	ผลลัพธ์	ระดับความสำคัญ
F01	ค้นหาข้อมูล	ผู้ใช้งานทั่วไป	คำค้นหา	รายการที่ตรงกับคำค้นหา	Core
F02	ดูรายละเอียด	ผู้ใช้งานทั่วไป	รายการที่เลือก	ข้อมูลรายละเอียด	Core
F03	บันทึกรายการโปรด	ผู้ใช้งาน	รหัสรายการ	รายการโปรดของผู้ใช้งาน	Supporting

ผลลัพธ์ที่ต้องส่งในส่วนที่ 1

1. ข้อมูลสรุปโครงงานและกลุ่มผู้ใช้งาน
2. รายการ Core Features, Supporting Features และ Future Features
3. ตารางรายละเอียดฟังก์ชัน
4. Use Case Diagram ฉบับปรับปรุงให้ตรงกับขอบเขตล่าสุด
5. คำอธิบายการเปลี่ยนแปลงจากข้อเสนอโครงงานเดิม หากมีการเพิ่ม ลด หรือปรับฟังก์ชัน

2. ออกแบบโครงสร้างข้อมูลสำหรับ Firebase

โครงงานทุกกลุ่มกำหนดให้ใช้ **Firebase Realtime Database** เป็นฐานข้อมูลหลัก นักศึกษาจึงต้องออกแบบโครงสร้างข้อมูลในรูปแบบ JSON ที่สามารถนำเข้า Firebase ได้จริง การมีไฟล์ JSON เพียงอย่างเดียวยังไม่เพียงพอ นักศึกษาต้องแสดงเหตุผลว่าโครงสร้างข้อมูลดังกล่าวรองรับฟังก์ชันและหน้าจอของระบบอย่างไร

2.1 กำหนดกลุ่มข้อมูลหลัก

ให้นักศึกษาวิเคราะห์ว่าระบบต้องจัดเก็บข้อมูลประเภทใดบ้าง เช่น

- categories
- items
- users
- favorites
- reviews
- settings

ชื่อ Node และ Field ให้ใช้ภาษาอังกฤษ ใช้ตัวอักษรพิมพ์เล็ก และกำหนดรูปแบบการตั้งชื่อให้สม่ำเสมอ เช่น snake_case หรือ camelCase อย่างใดอย่างหนึ่ง

2.2 ออกแบบโครงสร้าง JSON

ตัวอย่างโครงสร้างข้อมูลสำหรับ Firebase Realtime Database

```
{
  "categories": {
    "category_001": {
      "name": "อาหารภาคกลาง",
      "description": "รายการอาหารพื้นถิ่นในภาคกลาง",
      "image_url": "https://example.com/images/category_001.jpg"
    }
  },
  "foods": {
    "food_001": {
      "category_id": "category_001",
      "name_th": "ต้มยำกุ้ง",
      "name_en": "Tom Yum Goong",
      "description": "อาหารไทยประเภทยำที่มีรสเปรี้ยวและเผ็ด",
      "image_url": "https://example.com/images/food_001.jpg",
      "is_recommended": true
    }
  }
}
```

โครงสร้างนี้เป็นเพียงตัวอย่าง นักศึกษาต้องออกแบบให้เหมาะสมกับโครงงานของตนเอง ห้ามคัดลอกชื่อ Node และ Field ไปใช้โดยไม่วิเคราะห์

2.3 กำหนดรหัสข้อมูล

ข้อมูลแต่ละรายการต้องมีรหัสที่ไม่ซ้ำกัน เช่น

- category_001
- product_001
- place_001
- article_001

รหัสอ้างอิงต้องตรงกันทั้งในไฟล์ JSON รูปภาพ และข้อมูลที่เชื่อมโยงกัน เช่น ค่า category_id ในรายการอาหารต้องตรงกับรหัสที่มีอยู่ใน categories

2.4 จัดทำ Data Dictionary

Data Dictionary ต้องอธิบายโครงสร้างของแต่ละ Node และ Field

Node	Field	ชนิดข้อมูล	จำเป็น	คำอธิบาย	ตัวอย่าง
foods	name_th	String	ใช่	ชื่ออาหารภาษาไทย	ต้มยำกุ้ง
foods	name_en	String	ใช่	ชื่ออาหารภาษาอังกฤษ	Tom Yum Goong
foods	category_id	String	ใช่	รหัสหมวดหมู่	category_001
foods	image_url	String	ใช่	URL ของรูปภาพ	https://...
foods	is_recommended	Boolean	ไม่	สถานะรายการแนะนำ	true

2.5 เชื่อมโยงข้อมูลกับสินทรัพย์ดิจิทัล

ข้อมูลรูปภาพต้องเชื่อมโยงกับสินทรัพย์ดิจิทัลจากงานที่ 4 โดยกำหนดให้

- ชื่อไฟล์มีความสอดคล้องกับรหัสข้อมูล
- หลีกเลี่ยงชื่อไฟล์ภาษาไทยและช่องว่าง
- ใช้นามสกุลไฟล์อย่างสม่ำเสมอ เช่น .webp หรือ .jpg
- ใช้ URL หรือ Path ในรูปแบบเดียวกันทุก Record
- ห้ามใช้ Path ที่อ้างอิงตำแหน่งเฉพาะในเครื่องส่วนตัว เช่น C:\Users\...

ตัวอย่างความสัมพันธ์ระหว่างข้อมูลกับไฟล์ภาพ

รหัสข้อมูล: food_001

ชื่อไฟล์: food_001.webp

Field: image_url

หากยังไม่ได้ Upload รูปภาพ นักศึกษาสามารถกำหนดชื่อไฟล์หรือ Path ชั่วคราวได้ แต่ต้องออกแบบให้สามารถเปลี่ยนเป็น URL ได้ภายหลัง

2.6 ตรวจสอบคุณภาพไฟล์ JSON

ก่อนส่งงานต้องตรวจสอบว่า

- ไฟล์มีโครงสร้าง JSON ที่ถูกต้อง
- ไม่มีเครื่องหมาย comma เกินหรือตกหล่น
- Field ของข้อมูลประเภทเดียวกันใช้ชื่อเหมือนกัน
- ชนิดข้อมูลถูกต้อง เช่น Number, String, Boolean และ Array
- รหัสข้อมูลไม่ซ้ำกัน
- รหัสอ้างอิงเชื่อมโยงไปยังข้อมูลที่มีอยู่จริง
- ไม่มีข้อมูลทดลองที่ไม่เกี่ยวข้อง
- สามารถ Import เข้า Firebase Realtime Database ได้โดยไม่เกิดข้อผิดพลาด

Key ของ Firebase Realtime Database ต้องไม่มีอักขระ ., $, #, [,] และ /

ปริมาณข้อมูลขั้นต่ำ

แต่ละกลุ่มต้องจัดเตรียมข้อมูลจริงในปริมาณที่เพียงพอสำหรับแสดงผลและทดสอบระบบ โดยพิจารณาตามประเภทโครงงาน ทั้งนี้ควรมี

- ข้อมูลหมวดหมู่อย่างน้อย 3–5 หมวดหมู่
- ข้อมูลหลักอย่างน้อย 15–20 รายการ
- แต่ละรายการต้องมีข้อมูลครบตาม Field ที่กำหนด
- ต้องใช้ข้อมูลและภาพจริง ไม่ใช้ข้อความประเภท Lorem ipsum

ผลลัพธ์ที่ต้องส่งในส่วนที่ 2

1. แผนภาพโครงสร้างข้อมูลของ Firebase
2. Data Dictionary
3. ไฟล์ข้อมูลนามสกุล .json
4. ตารางแสดงความสัมพันธ์ระหว่างข้อมูล
5. รายการไฟล์ภาพหรือสินทรัพย์ดิจิทัลที่เชื่อมโยงกับข้อมูล
6. หลักฐานการตรวจสอบความถูกต้องของ JSON
7. ภาพหน้าจอผลการ Import JSON เข้า Firebase Realtime Database

3. ออกแบบโครงสร้างหน้าจอและการนำทาง

การออกแบบส่วนนี้ควรดำเนินการตามลำดับจากภาพรวมไปสู่รายละเอียด ได้แก่

1. จัดทำรายการหน้าจอ
2. จัดทำ Sitemap
3. ออกแบบ User Flow
4. จัดทำ Wireframe ของหน้าจอหลัก
5. จัดทำ Wireframe ของหน้าจอสำคัญ
6. ตรวจสอบความสอดคล้องกับฟังก์ชันและข้อมูล

ไม่ควรเริ่มวาด Wireframe ทันทีโดยยังไม่กำหนดรายการหน้าจอและเส้นทางการใช้งาน เพราะอาจทำให้หน้าจอซ้ำซ้อนหรือขาดหน้าจอที่จำเป็น

3.1 จัดทำรายการหน้าจอ

ให้นักศึกษาระบุหน้าจอที่จำเป็นต่อการทำงานของระบบ

รหัสหน้าจอ	ชื่อหน้าจอ	วัตถุประสงค์	ฟังก์ชันที่เกี่ยวข้อง	ข้อมูลที่ใช้
P01	Home	แสดงภาพรวมของระบบ	F01, F02	categories, items
P02	Item List	แสดงรายการตามหมวดหมู่	F02	items
P03	Item Detail	แสดงรายละเอียด	F02, F03	items, favorites
P04	Search	ค้นหาข้อมูล	F01	items

3.2 จัดทำ Sitemap

Sitemap ใช้แสดงโครงสร้างลำดับชั้นของหน้าจอทั้งหมด เช่น

```
Home
├── Categories
│   ├── Item List
│   └── Item Detail
├── Search
│   └── Search Results
├── Favorites
│   └── Item Detail
└── About
```

Sitemap ต้องแสดงให้เห็นว่า

- ระบบมีหน้าจอใดบ้าง
- หน้าจอใดเป็นหน้าจอหลัก
- หน้าจอใดอยู่ภายใต้หมวดหมู่เดียวกัน

- ผู้ใช้งานสามารถเข้าถึงแต่ละหน้าได้จากส่วนใด

3.3 จัดทำ User Flow

User Flow แสดงลำดับการใช้งานเพื่อให้ผู้ใช้บรรลุเป้าหมายอย่างใดอย่างหนึ่ง เช่น

Home ⟶ เลือกหมวดหมู่ ⟶ แสดงรายการ ⟶ เลือกรายการ ⟶ แสดงรายละเอียด

แต่ละกลุ่มต้องจัดทำ User Flow อย่างน้อย 2–3 กระบวนการ โดยเลือกจาก Core Features เช่น

- การค้นหาและดูรายละเอียด
- การเลือกหมวดหมู่และเปิดรายการ
- การเพิ่มหรือลบรายการโปรด
- การเพิ่มหรือแก้ไขข้อมูล
- การเข้าสู่ระบบและจัดการข้อมูล

3.4 จัดทำ Wireframe ของหน้าจอหลัก

Wireframe ของหน้าจอหลักหรือ Home Page ต้องแสดงโครงสร้างสำคัญ เช่น

- Header
- Logo หรือชื่อระบบ
- Navigation
- Search
- Hero Section หรือส่วนแนะนำระบบ
- Category Section
- Featured Items หรือรายการแนะนำ
- Footer

Wireframe ต้องระบุชื่อส่วนประกอบให้ชัดเจน ไม่ควรวาดเป็นกล่องเปล่าโดยไม่มีคำอธิบาย

3.5 จัดทำ Wireframe ของหน้าจอสำคัญ

นอกจาก Home Page แล้ว ต้องออกแบบ Wireframe ของหน้าจอที่เกี่ยวข้องกับ Core Features เช่น

- Category หรือ Item List
- Item Detail

- Search Results
- Favorites
- Form สำหรับเพิ่มหรือแก้ไขข้อมูล
- Login หรือ Profile ถ้าอยู่ในขอบเขตโครงงาน

แต่ละกลุ่มควรจัดทำ Wireframe อย่างน้อย 4–6 หน้าจอ โดยจำนวนจริงขึ้นอยู่กับขอบเขตของโครงงาน

3.6 ระบุพฤติกรรมของหน้าจอ

Wireframe แต่ละหน้าต้องมีคำอธิบายประกอบ ได้แก่

- วัตถุประสงค์ของหน้าจอ
- ข้อมูลที่นำมาแสดง
- การดำเนินการที่ผู้ใช้สามารถทำได้
- หน้าจอปลายทางเมื่อกดปุ่มหรือลิงก์
- สิ่งที่แสดงเมื่อไม่มีข้อมูล
- สิ่งที่แสดงระหว่างรอข้อมูล
- สิ่งที่แสดงเมื่อเกิดข้อผิดพลาด

3.7 ตรวจสอบความสอดคล้อง

นักศึกษาต้องตรวจสอบว่า

- Core Feature ทุกฟังก์ชันมีหน้าจอรองรับ
- หน้าจอทุกหน้ามีเส้นทางเข้าถึง
- ปุ่มและลิงก์มีจุดหมายที่ชัดเจน
- ข้อมูลที่แสดงใน Wireframe มีอยู่ในโครงสร้าง JSON
- ไม่ออกแบบข้อมูลบนหน้าจอที่ไม่มี Field รองรับในฐานข้อมูล
- ใช้รูปแบบ Header, Navigation, Card และ Footer อย่างสม่ำเสมอ

ผลลัพธ์ที่ต้องส่งในส่วนที่ 3

1. ตารางรายการหน้าจอ
2. Sitemap
3. User Flow อย่างน้อย 2–3 กระบวนการ
4. Wireframe ของ Home Page
5. Wireframe ของหน้าจอสำคัญอย่างน้อย 4–6 หน้าจอ

6. คำอธิบายการทำงานของแต่ละหน้าจอ
7. ตารางเชื่อมโยงฟังก์ชัน หน้าจอ และข้อมูล

4. ออกแบบ Component และโครงสร้างโครงการ React

ให้นักศึกษาวิเคราะห์ Wireframe ที่จัดทำในส่วนที่ 3 แล้วแบ่งส่วนติดต่อผู้ใช้ออกเป็น React Component

การออกแบบ Component ไม่ควรพิจารณาเฉพาะ Wireframe ของ Home Page แต่ต้องพิจารณา Wireframe ของหน้าจอสำคัญทุกหน้า เพื่อค้นหาส่วนที่ใช้ร่วมกันและส่วนที่สามารถนำกลับมาใช้ซ้ำได้

4.1 วิเคราะห์ Component จาก Wireframe

ให้นักศึกษาทำเครื่องหมายหรือกำหนดกรอบบน Wireframe เพื่อแสดงว่าแต่ละบริเวณจะพัฒนาเป็น Component ใด เช่น

- Header
- Navbar
- SearchBar
- CategoryList
- CategoryCard
- ItemList
- ItemCard
- ItemDetail
- FavoriteButton
- Footer

ตัวอย่างการวิเคราะห์ Home Page

```
App
├── Header
│   ├── Logo
│   └── Navbar
├── SearchBar
├── CategoryList
│   └── CategoryCard
├── FeaturedItemList
│   └── ItemCard
└── Footer
```

ตัวอย่างการวิเคราะห์หน้า Detail

```
ItemDetailPage
```

```
├── Header
├── Breadcrumb
├── ItemImage
├── ItemInformation
├── FavoriteButton
├── RelatedItemList
│   └── ItemCard
└── Footer
```

4.2 จำแนกประเภทของ Component

ให้นักศึกษาแบ่ง Component เป็นกลุ่ม ดังนี้

ประเภท	หน้าที่	ตัวอย่าง
Layout Component	กำหนดโครงสร้างที่ใช้ร่วมกัน	Header, Navbar, Footer
Page Component	เป็นหน้าหลักที่เชื่อมกับ Route	HomePage, DetailPage
Reusable UI Component	ส่วนติดต่อที่นำกลับมาใช้ซ้ำได้	Button, Card, SearchBar
Feature Component	รองรับฟังก์ชันเฉพาะ	FavoriteButton, FilterPanel
Data Display Component	แสดงข้อมูลจากฐานข้อมูล	ItemList, ItemDetail

4.3 จัดทำ Component Hierarchy

ให้จัดทำแผนภาพแสดงความสัมพันธ์ระหว่าง Page และ Component อย่างน้อยสำหรับหน้าจอสำคัญทุกหน้า

Component ที่มีลักษณะเหมือนกันควรออกแบบให้ใช้ร่วมกัน เช่น ItemCard ควรใช้ได้ทั้งในหน้า Home, Category, Search Results และ Favorites โดยเปลี่ยนข้อมูลผ่าน Props

4.4 จัดทำ Component Specification

Component	หน้าที่	ใช้ในหน้าจอ	Props	State	ข้อมูลจาก Firebase
ItemCard	แสดงข้อมูลสรุปของรายการ	Home, List, Search	item, onSelect	ไม่มี	items
SearchBar	รับคำค้นหาจากผู้ใช้	Home, Search	onSearch	keyword	ไม่มี
ItemList	แสดงรายการหลายรายการ	List, Search	items	loading	items
FavoriteButton	เพิ่มหรือลบรายการโปรด	Detail	itemId	isFavorite	favorites

นักศึกษาต้องอธิบายอย่างน้อยว่า

- Component ทำหน้าที่อะไร
- Component ปรากฏในหน้าจอใด
- รับค่าใดผ่าน Props
- จัดการ State ใด
- ใช้ข้อมูลจาก Node ใดใน Firebase
- มีเหตุการณ์ใดที่ผู้ใช้สามารถกระทำได้

4.5 ออกแบบ Routing

จัดทำตารางเส้นทางของเว็บแอปพลิเคชัน

Path	Page Component	หน้าที่
/	HomePage	แสดงหน้าแรก
/categories	CategoryPage	แสดงหมวดหมู่
/items/:id	ItemDetailPage	แสดงรายละเอียด
/search	SearchPage	แสดงผลการค้นหา
/favorites	FavoritePage	แสดงรายการโปรด

Path ต้องสัมพันธ์กับ Sitemap และ User Flow ที่ออกแบบไว้

4.6 ออกแบบโครงสร้าง Folder ของ React Project

ตัวอย่างโครงสร้างโครงการ

```
src/
├── assets/
│   └── images/
├── components/
│   ├── layout/
│   ├── common/
│   └── features/
├── pages/
├── services/
│   └── firebaseService.js
├── hooks/
├── data/
├── routes/
├── styles/
├── App.jsx
└── main.jsx
```

นักศึกษาสามารถปรับโครงสร้างได้ตามความเหมาะสม แต่ต้องอธิบายหน้าที่ของแต่ละ Folder และต้องไม่สร้าง Folder ที่ไม่มีการใช้งานจริง

4.7 ตรวจสอบการออกแบบ Component

ก่อนส่งงานให้ตรวจสอบว่า

- Wireframe ทุกหน้าสามารถแปลงเป็น Component ได้
- Component ที่ใช้ซ้ำได้รับการแยกออกจาก Page Component
- ไม่สร้าง Component ขนาดใหญ่ที่รับผิดชอบทุกอย่างในไฟล์เดียว
- ไม่แยก Component ย่อยมากเกินไปจนโครงสร้างซับซ้อนโดยไม่จำเป็น
- ชื่อ Component ใช้รูปแบบ PascalCase
- Page Component สอดคล้องกับ Route
- Props และ State ที่กำหนดมีเหตุผลรองรับ
- Component ที่แสดงข้อมูลเชื่อมโยงกับ Node และ Field ใน Firebase ได้จริง

ผลลัพธ์ที่ต้องส่งในส่วนที่ 4

1. Wireframe ที่กำหนดขอบเขตของ Component
2. Component Hierarchy ของหน้าจอสำคัญ
3. ตารางจำแนกประเภท Component
4. Component Specification
5. ตาราง Routing
6. โครงสร้าง Folder ของ React Project
7. คำอธิบายเหตุผลในการแบ่ง Component

ตารางแสดงความสัมพันธ์ของผลลัพธ์

นักศึกษาต้องแสดงให้เห็นว่าผลลัพธ์ทั้ง 4 ส่วนมีความสัมพันธ์กัน ไม่ใช่เอกสารที่แยกออกจากกัน

ฟังก์ชัน	ข้อมูล Firebase	หน้าจอ	React Component
แสดงหมวดหมู่	categories	Home, Category	CategoryList, CategoryCard
แสดงรายการ	items	Home, Item List	ItemList, ItemCard
ดูรายละเอียด	items/{id}	Item Detail	ItemDetail
ค้นหาข้อมูล	items	Search Results	SearchBar, SearchResultList

บันทึกรายการโปรด	favorites	Detail, Favorites	FavoriteButton, FavoriteList

รูปแบบการส่งงาน

ให้แต่ละกลุ่มส่งผลงานดังต่อไปนี้

1. รายงานงานที่ 5 ในรูปแบบ PDF จำนวน 1 ไฟล์
2. ไฟล์ข้อมูลสำหรับ Firebase นามสกุล .json จำนวน 1 ไฟล์
3. ไฟล์ต้นฉบับ Wireframe หรือ Link ที่ผู้สอนสามารถเปิดดูได้
4. รูปภาพ Sitemap และ User Flow
5. โฟลเดอร์สินทรัพย์ดิจิทัลฉบับปรับปรุง
6. ไฟล์ README.md อธิบายรายการไฟล์และวิธีตรวจสอบผลงาน
7. บีบอัดไฟล์ทั้งหมดเป็นไฟล์ .zip ก่อนส่ง

รูปแบบชื่อไฟล์

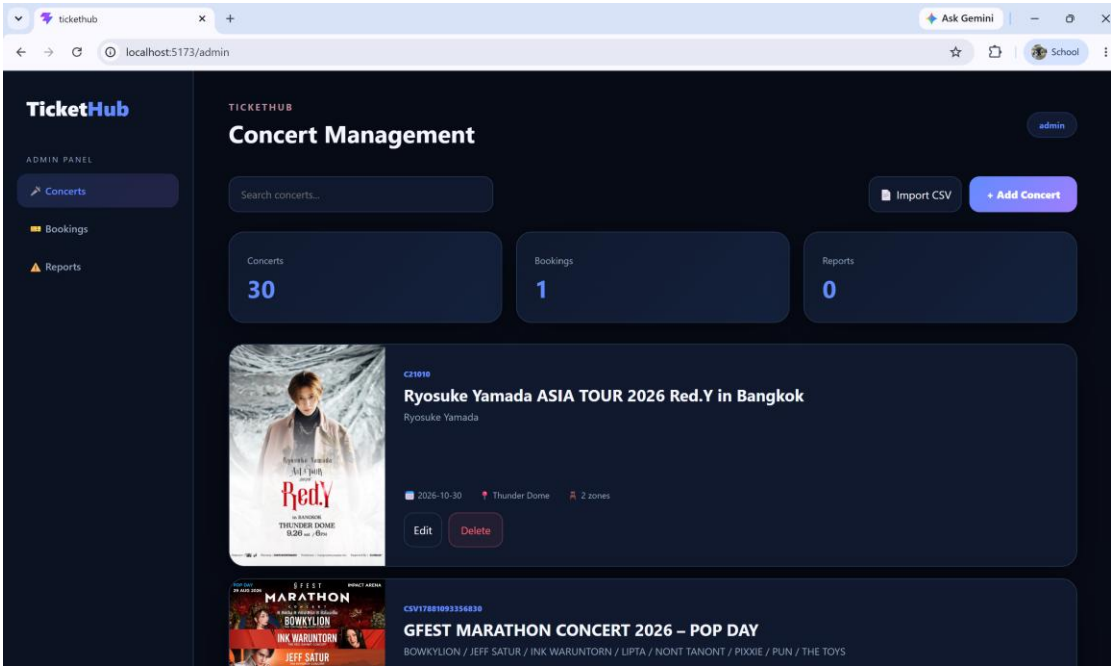
GroupXX_ProjectName_Assignment05.zip

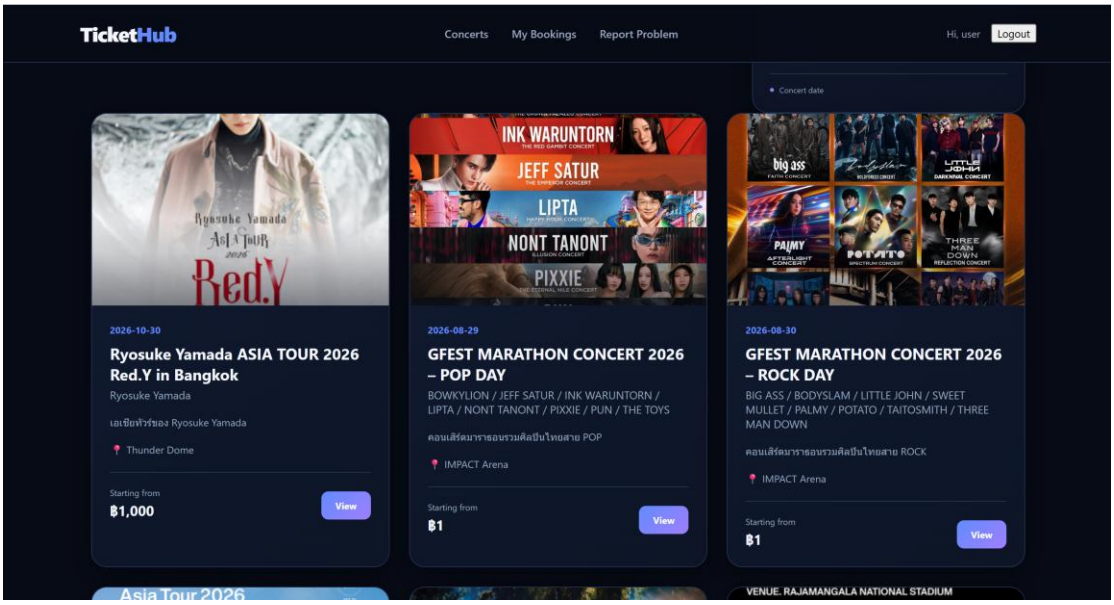
ตัวอย่าง

Group03_LocalTravelGuide_Assignment05.zip

ข้อกำหนดสำคัญ

- สมาชิกทุกคนต้องมีส่วนร่วมในการวิเคราะห์และออกแบบ
- ข้อมูลใน JSON ต้องเป็นข้อมูลจริงและสามารถนำไปใช้พัฒนาโครงงานได้
- Wireframe ต้องออกแบบตามฟังก์ชันของโครงงาน ไม่ใช้ Template โดยไม่มีการปรับให้เหมาะสม
- Component ต้องวิเคราะห์จาก Wireframe ของโครงงานจริง
- ผลลัพธ์ทั้ง 4 ส่วนต้องเชื่อมโยงและสอดคล้องกัน
- สามารถใช้ Generative AI เป็นผู้ช่วยเสนอแนวคิดหรือตรวจสอบงานได้ แต่นักศึกษาต้องสามารถอธิบายเหตุผลของการออกแบบทุกส่วนได้
- หากนักศึกษาไม่สามารถอธิบายโครงสร้างข้อมูล Wireframe หรือ Component ที่ส่งมาได้ จะถือว่ายังไม่ผ่านการตรวจสอบงาน


tickethub
Ask Gemini
localhost:5173/admin
School
TicketHub
ADMIN PANEL
Concerts
Bookings
Reports
TICKETHUB
Concert Management
admin
Search concerts...
Import CSV
+ Add Concert
Concerts
30
Bookings
1
Reports
0
C21010
Ryosuke Yamada ASIA TOUR 2026 Red.Y in Bangkok
Ryosuke Yamada
2026-10-30
Thunder Dome
2 zones
Edit
Delete
CSV17881093356830
GFEST MARATHON CONCERT 2026 – POP DAY
BOWKYLION / JEFF SATUR / INK WARUNTORN / LIPTA / NONT TANONT / PIXXIE / PUN / THE TOYS


TicketHub
Concerts
My Bookings
Report Problem
Hi, user
Logout
Concert date
2026-10-30
Ryosuke Yamada ASIA TOUR 2026 Red.Y in Bangkok
Ryosuke Yamada
เอเชียทัวร์ของ Ryosuke Yamada
Thunder Dome
Starting from
฿1,000
View
2026-08-29
GFEST MARATHON CONCERT 2026 – POP DAY
BOWKYLION / JEFF SATUR / INK WARUNTORN / LIPTA / NONT TANONT / PIXXIE / PUN / THE TOYS
คอนเสิร์ตมาราธอนรวมศิลปินไทยสาย POP
IMPACT Arena
Starting from
฿1
View
2026-08-30
GFEST MARATHON CONCERT 2026 – ROCK DAY
BIG ASS / BODYSLAM / LITTLE JOHN / SWEET MULLET / PALMY / POTATO / TAITOSMITH / THREE MAN DOWN
คอนเสิร์ตมาราธอนรวมศิลปินไทยสาย ROCK
IMPACT Arena
Starting from
฿1
View
Asia Tour 2026
VENUE. RAJAMANGALA NATIONAL STADIUM